УНИВЕРСИТЕТ ИТМО

ОТЧЁТ
по лабораторной работе № 3

Исследование конвейера процессора SCR1

Вариант 3. Инструкция ADD

Выполнил:
Дворкин Борис Александрович
Группа P4130

Санкт-Петербург
2026

1. Цель работы

Изучить выполнение инструкции ADD в конвейере SCR1. Добавить в тестбенч модуль, который при выполнении этой инструкции выводит её атрибуты.

2. Выполнение работы

В списке тестов оставлен isa/rv32ui/add.S. Для симуляции выбраны конфигурация MAX и интерфейс AHB. Использованы Verilator 5.054, собранный из исходников, и RISC-V GCC 14.2.0.

Добавлен модуль src/tb/scr1_tb_log_cmd.sv. Он сохраняет машинное слово при передаче команды на исполнение и печатает информацию при её завершении. ADD определяется по opcode, funct3 и funct7. Последнее поле позволяет отличить её от SUB и MUL.

Код модуля:

```
module scr1_tb_log_cmd (
    input logic        clk,
    input logic        rst_n,
    input logic        id_accept,
    input logic [31:0] id_instr,
    input logic        ex_retire,
    input logic        ex_exception,
    input logic [31:0] curr_pc,
    input logic [31:0] rs1_value,
    input logic [31:0] rs2_value,
    input logic [31:0] rd_value
);
    logic [31:0] ex_instr;
    wire [6:0] opcode = ex_instr[6:0];
    wire [4:0] rd     = ex_instr[11:7];
    wire [2:0] funct3 = ex_instr[14:12];
    wire [4:0] rs1    = ex_instr[19:15];
    wire [4:0] rs2    = ex_instr[24:20];
    wire [6:0] funct7 = ex_instr[31:25];

    wire is_add = (opcode == 7'b0110011)
               && (funct3 == 3'b000)
               && (funct7 == 7'b0000000);
    wire add_retire = rst_n && ex_retire && !ex_exception && is_add;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            ex_instr <= 32'b0;
        end else begin
            if (id_accept)
                ex_instr <= id_instr;
            if (add_retire) begin
                $display({
                    "ADD pc=%08h instr=%08h opcode=%02h funct3=%01h funct7=%02h rd=x%0d",
                    " rs1=x%0d rs2=x%0d rs1_value=%08h rs2_value=%08h result=%08h"
                },
                         curr_pc, ex_instr, opcode, funct3, funct7,
                         rd, rs1, rs2, rs1_value, rs2_value, rd_value);
            end
        end
    end
endmodule
```

3. Подключение и запуск

В src/tb/scr1_top_tb_ahb.sv перед endmodule добавлено подключение наблюдателя:

```
scr1_tb_log_cmd i_lab3_log (
    .clk          (clk),
    .rst_n        (i_top.i_core_top.i_pipe_top.pipe_rst_n),
    .id_accept    (i_top.i_core_top.i_pipe_top.idu2exu_req
                && i_top.i_core_top.i_pipe_top.exu2idu_rdy),
    .id_instr     (i_top.i_core_top.i_pipe_top.ifu2idu_instr),
    .ex_retire    (i_top.i_core_top.i_pipe_top.instret),
    .ex_exception (i_top.i_core_top.i_pipe_top.exu_exc_req),
    .curr_pc      (i_top.i_core_top.i_pipe_top.curr_pc),
    .rs1_value    (i_top.i_core_top.i_pipe_top.mprf2exu_rs1_data),
    .rs2_value    (i_top.i_core_top.i_pipe_top.mprf2exu_rs2_data),
    .rd_value     (i_top.i_core_top.i_pipe_top.exu2mprf_rd_data)
);
```

В список src/ahb_tb.files добавлена строка:

```
tb/scr1_tb_log_cmd.sv
```

В sim/tests/riscv_isa/rv32_tests.inc оставлен тест:

```
rv32_isa_tests += isa/rv32ui/add.S
```

Команда запуска из корня репозитория:

```
make clean
make run_verilator_wf TARGETS=riscv_isa BUS=AHB CFG=MAX TRACE=1
```

Результат симуляции:

```
Summary: 1/1 tests passed
add.hex    OK    PASS
```

Пример вывода наблюдателя для команды по адресу 0x2d8:

```
ADD pc=000002d8 instr=002081b3 opcode=33 funct3=0 funct7=00
    rd=x3 rs1=x1 rs2=x2
    rs1_value=00000003 rs2_value=00000007 result=0000000a
```

В исходном логе это одна строка. Инструкция add x3, x1, x2 складывает значения 3 и 7 и записывает 10 в x3. Все 52 записи наблюдателя проверены скриптом report/check_debug.py: адреса и коды сверены с дизассемблером, результаты сложения совпали. Тесты SUB и MUL также прошли без ложных сообщений ADD.

4. Временная диаграмма

Сохранённая трасса открыта в GTKWave. На рисунке показаны сигналы выборки инструкции, её код на декодировании и исполнении, PC, операнды и результат.

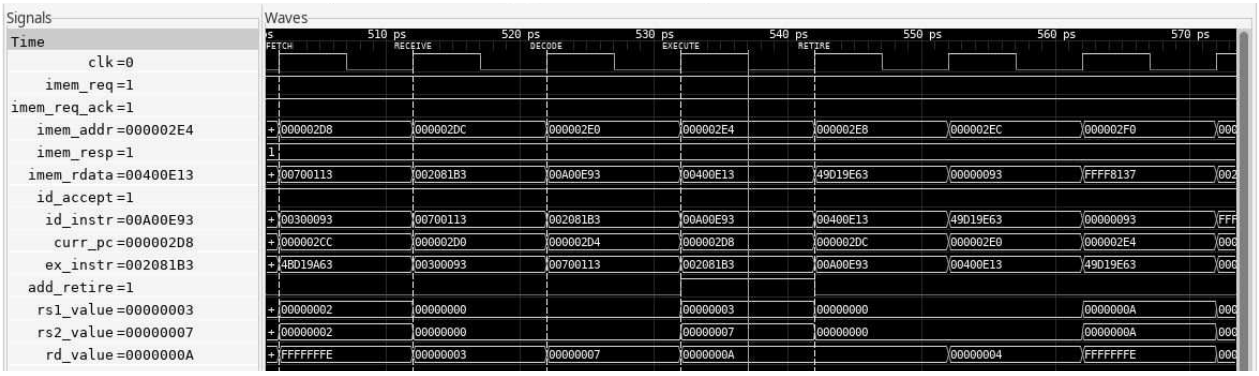


Рисунок 1. Выполнение ADD по адресу 0x2d8 (фрагмент окна GTKWave).

На интервале 502-512 ps запрашивается инструкция по адресу 0x2d8. На 512-522 ps из памяти приходит слово 0x002081b3. На 522-532 ps оно находится на декодировании, а на 532-542 ps выполняется. Операнды равны 3 и 7, результат равен 0x0a. На фронте 542 ps команда завершается.

На маркере 537 ps curr_pc равен 0x2d8, а imem_addr равен 0x2e4. Исполнение ADD идёт одновременно с выборкой более поздней команды. Это показывает работу конвейера. Время на рисунке относится к модели симуляции, а не к измерению частоты реального процессора.

Скриншот находится в report/waveform.png, вывод модуля - в report/debug.log. Для повторного просмотра сохранены add.vcd и add.gtkw:

```
gtkwave report/add.vcd report/add.gtkw
```

5. Вывод

В работе прослежено выполнение ADD через четыре стадии конвейера SCR1. В тестбенч добавлен модуль, который выводит код и атрибуты инструкции, её адрес, операнды и результат. Тест ADD завершился успешно. По диаграмме видно, как команда проходит от запроса памяти до исполнения и как разные стадии одновременно обрабатывают разные команды.

Репозиторий: https://github.com/Imtjl/scr1
Ветка: lab_scr1_pipe_analysis.